

Here is the improved roadmap as a clean, copy-ready document:

QA ENGINEERING INTENSIVE: 2-WEEK ROADMAP

From Software Engineer to QA Expert

OVERVIEW

Goal: In 14 days, you will be able to think, work, and communicate like a professional QA Engineer — with real skills in manual testing, API testing, SQL validation, test automation, and portfolio presentation.

Philosophy: Depth over speed. One thing understood well beats five things copied blindly.

FINAL DELIVERABLES

Documentation

- Test Strategy Document
- Test Plan Document
- Requirement Analysis Notes
- 100+ Test Cases
- 20+ Bug Reports

Testing Artifacts

- Postman Collection (with auth, variables, and assertions)
- SQL Validation Queries
- API Automation Framework (Pytest)

- UI Automation Framework (Playwright)
- CI/CD Pipeline (GitHub Actions)

Portfolio

GitHub Repository: [qa-engineering-portfolio](#)

```
qa-engineering-portfolio/  
|  
├── requirements-analysis/  
├── test-strategy/  
├── test-plan/  
├── test-cases/  
├── bug-reports/  
├── postman/  
├── sql/  
├── api-automation/  
├── ui-automation/  
├── github-actions/  
└── README.md
```

WEEK 1: QA MINDSET + MANUAL TESTING + TEST DESIGN

Goal: Learn how professional testers think. No automation yet. Build a strong foundation in quality engineering that you can defend in any interview.

DAY 1 — QA Mindset + First Real Exploratory Session

Study

- What is QA? What is Testing?
- QA vs QC
- Verification vs Validation
- Defect vs Bug vs Failure
- Shift Left Testing and Shift Right Testing

- The Test Pyramid
- SDLC: Requirements → Design → Development → Testing → Deployment → Maintenance
- STLC: Requirement Analysis → Test Planning → Test Design → Execution → Reporting → Closure

Exercise

Choose one of the following real applications:

- WhatsApp Web
- Instagram Web
- LinkedIn

Do both of the following:

1. Write 50 possible failure scenarios from memory
2. Open the application and spend 45 minutes actually exploring it. Note 10 things you assumed were working that you had not consciously verified

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`day01-failure-analysis.md`

DAY 2 — Test Design Techniques

Study

1. Equivalence Partitioning
2. Boundary Value Analysis
3. Decision Table Testing
4. State Transition Testing
5. Error Guessing
6. Risk-Based Testing

Exercise

Feature: User Registration Form

For each of the 6 techniques above:

- Write at least 5 test scenarios applying that specific technique
- Label each scenario with the technique used

Total: 30 test scenarios minimum

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`day02-test-design.xlsx`

DAY 3 — Requirement Analysis

Study

- How to read and review requirements professionally
- How to identify:
 - Missing requirements
 - Ambiguous requirements
 - Contradictory requirements
- Why unclear requirements are the root cause of most bugs

Exercise

Features to analyse: Login, Registration, Password Reset

For each feature:

- Write 15–20 clarification questions a QA engineer would ask before testing begins
- Categorise each question: Missing / Ambiguous / Contradictory

Total: 50 questions minimum

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`day03-requirement-review.md`

DAY 4 — Test Case Design

Study

Test Case Structure — Required Fields:

Field	Description
Test Case ID	Unique identifier
Title	Clear one-line summary
Preconditions	What must be true before the test
Test Steps	Numbered, specific actions
Expected Result	Exact outcome if software works correctly
Actual Result	What actually happened
Status	Pass / Fail / Blocked

Exercise

Write 50 detailed test cases covering:

- Registration (20 cases)
- Login (20 cases)
- User Profile (10 cases)

Each test case must be complete — no skipping fields.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`test-cases-v1.xlsx`

DAY 5 — Bug Reporting

Study

Severity Levels:

- Blocker — System cannot be used at all
- Critical — Core feature is broken
- Major — Important feature is impaired
- Minor — Small issue with low impact

Priority Levels:

- High — Must be fixed immediately
- Medium — Should be fixed in current cycle
- Low — Can be deferred

A good bug report answers:

- What happened?
- How do I reproduce it?
- What did I expect?
- What environment was it in?

Real-World Research

Go to GitHub and find 10 real bug reports in open-source projects (try VS Code, Firefox, or Playwright repositories). For each one, study:

- How the root cause was described
- How reproduction steps were written
- How severity and impact were communicated

Exercise

Create 20 sample bug reports. At least 5 must be inspired by real bugs you observed during your Day 1 exploratory session.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`bug-reports-v1.xlsx`

DAY 6 — SQL for QA

Study

SQL Commands Every QA Engineer Must Know:

SELECT
WHERE
ORDER BY
GROUP BY
COUNT
INNER JOIN
LEFT JOIN

Why SQL matters for QA: When you click a button in a UI, data is written to a database. SQL lets you verify the data actually landed correctly — not just that the screen looked right.

Exercise

Create three tables:

- Users
- Orders
- Products

Write 30 SQL validation queries that answer real QA questions, such as:

- Did the registration create a user record with all required fields?
- Did placing an order reduce product inventory?
- Are there any orphaned orders with no associated user?
- Are there duplicate email addresses in the users table?

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`sql-validation.sql`

DAY 7 — REST API Concepts + First Postman Requests

Study

HTTP Methods:

- GET — Retrieve data
- POST — Create data
- PUT — Replace data entirely
- PATCH — Update data partially
- DELETE — Remove data

Status Codes You Must Know:

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized

403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Entity
500	Internal Server Error

Also study: Headers, JWT tokens, Authentication vs Authorization

Exercise

Using JSONPlaceholder (<https://jsonplaceholder.typicode.com>):

- Send at least one request for each HTTP method
- Verify the correct status code is returned
- Inspect the response body and headers
- Write notes on what each response tells you as a tester

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`day07-api-testing-notes.md`

WEEK 2: API TESTING + AUTOMATION + PORTFOLIO

Goal: Build real, working automation you can explain — not copy-paste code you cannot defend. Every line of code you write this week, make sure you understand why it is there.

DAY 8 — Postman: Collections, Environments, and Assertions

Study

- Collections and how to organise requests
- Environments and variables (base URL, tokens, user IDs)
- Pre-request scripts
- Test scripts and assertions
- How to chain requests (use response data from one request in the next)

Exercise

Build a complete Postman collection for the Users API on JSONPlaceholder covering:

- Get all users
- Get a single user
- Create a user
- Update a user
- Delete a user

For every request, write at least 3 assertions in the Tests tab. Examples:

- Status code is correct
- Response time is under 500ms
- Response body contains expected fields

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`users-api-postman-collection.json`

DAY 9 — Postman: Auth Flows + Error Scenarios

Study

- How to handle Bearer token authentication in Postman
- How to test unauthorised and forbidden scenarios
- How to test with invalid, missing, and malformed data
- How to use Postman environments to switch between test data sets

Exercise

Extend your Day 8 collection to include:

- At least 5 negative test cases (invalid data, missing fields, wrong auth)
- An environment with variables for base URL and any auth tokens
- A test that validates error messages in the response body — not just status codes

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

Updated `users-api-postman-collection.json`

DAY 10 — Pytest Fundamentals

Install

`pip install pytest requests`

Study

- How assertions work in Pytest
- Fixtures and why they exist
- Parametrize for data-driven tests
- Marks for categorising tests

Exercise

Write a simple API test suite for JSONPlaceholder Users API. Focus on:

- One test file with 10 well-written tests

- At least one parametrized test
- At least one fixture

Rule for today: Write fewer tests and understand them fully. Do not copy and paste anything you cannot explain.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`api-automation-v1/`

DAY 11 — Professional Pytest Framework

Study

Project Structure:

```
api-automation/  
├── tests/  
├── fixtures/  
├── utils/  
├── data/  
└── conftest.py
```

- How conftest.py works and why it is useful
- Setup and teardown
- How to generate HTML reports with pytest-html

Exercise

Refactor your Day 10 test suite into a professional structure:

- Move reusable logic into utils
- Move test data into a data folder
- Use conftest.py for shared fixtures

- Generate a test report

The goal is not more tests — it is better organised, more maintainable tests.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`api-automation/` (refactored)

DAY 12 — Playwright UI Automation

Install

```
pip install playwright  
playwright install
```

Study

- How to identify elements with locators
- Assertions in Playwright
- How to handle waits correctly (avoid hard-coded sleeps)
- How to capture screenshots on failure

Exercise

Choose a real public web application (the Todo app at <https://todomvc.com> is a good choice).

Automate exactly three flows — but do them properly:

1. Add a new item
2. Mark an item as complete
3. Delete an item

For each flow: use good locators, add assertions, and handle any waits explicitly.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

`ui-automation-v1/`

DAY 13 — CI/CD for QA with GitHub Actions

Study

- What Continuous Integration means for a QA engineer
- How GitHub Actions workflows are structured
- How to read a failing pipeline and diagnose what broke

Exercise

Create `.github/workflows/tests.yml` that:

- Triggers on every push to main
- Installs dependencies
- Runs your Pytest suite
- Runs your Playwright tests
- Uploads test reports as artifacts

Success criteria: Push a commit and watch your tests run automatically. Fix any failures until the pipeline is green.

Daily Reflection

Write two sentences:

- What did I test today?
- What did I learn about the software from testing it?

Deliverable

Working CI pipeline in your repository

DAY 14 — Final Portfolio Project + Interview Preparation

Application

Choose one:

- Student Management System (mock or public demo)
- Todo Application (<https://todomvc.com>)

Create

- Test Strategy Document
- Test Plan Document
- 100 Test Cases
- 20 Bug Reports
- SQL Validation Queries
- Postman Collection
- Pytest Framework
- Playwright Framework
- GitHub Actions Pipeline
- README.md that explains your entire portfolio clearly

Interview Preparation

For each topic below, practice explaining it out loud in your own words — not from memory, from understanding:

- SDLC and STLC
- Shift Left Testing
- Test Pyramid
- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Exploratory Testing
- Severity vs Priority
- API Testing
- SQL for Data Validation
- Postman
- Pytest
- Playwright
- CI/CD in Testing
- Test Strategy vs Test Plan

If you can explain every item above and show your portfolio repository, you will be ahead of the majority of entry-level QA candidates.

Final Deliverable

[qa-engineering-portfolio](#) — complete, clean, and ready to share

DAILY HABITS (EVERY DAY, ALL 14 DAYS)

Habit 1 — Edge Case Thinking

Pick one feature. Generate 30 edge cases for it.

Good features to use:

- Login
- Registration
- Search
- Checkout
- File Upload
- Password Reset

Habit 2 — Read Real Bug Reports

Go to GitHub Issues on any active open-source project. Read 3 bug reports. For each one, study:

- How was the root cause identified?
- How were the reproduction steps written?
- What was the severity and why?

Habit 3 — Question Everything

Open any real application. Ask:

- What can fail here?
- What assumptions is this interface making about the user?
- What edge cases are not being handled?
- What happens if the network is slow or drops entirely?

Habit 4 — Daily Reflection (Most Important)

Every single day, write two sentences in a running notes file:

- What did I test today?
- What did I learn about the software from testing it?

This single habit separates testers who find bugs from testers who improve product quality.

KEY IMPROVEMENTS IN THIS PLAN VS THE ORIGINAL

Area	Original Plan	This Plan
Exploratory Testing	Day 6 only	Day 1 — starts immediately
SQL	Abstract queries	Real data validation scenarios
API Testing	Week 2 only	Introduced Day 7 so Week 2 is less rushed
Postman	One day	Two days with auth and error scenarios
Automation	4 days for everything	Focused and manageable with real understanding
Daily Habits	Vague	Structured with a specific daily reflection
Interview Prep	Topic list only	Active verbal practice required
